

Software Verification & Validation

SVV Project Report

Submitted to:

Ma'am Jawaria Ramzan

Submitted by:

1. M. Saad Asad

[FA23-BSSE-4A- 017]

2. Abdul Rehman

[FA23-BSSE-4A- 008]



Department of Software Engineering

Lahore Garrison University

Microservices Order Processing System

(Event-Driven Formal Verification Model)

1. INTRODUCTION

1.1 SVV Overview

Software Verification and Validation (SVV) is a process used to ensure that a software system is correct, consistent, and behaves according to its requirements. Verification ensures that the system is built correctly, while validation ensures that the correct system is built according to user needs.

In this project, SVV techniques are applied to a Microservices Order Processing System using formal methods such as Z Notation, VDM, and Alloy Analyzer. These techniques help in mathematically verifying system behavior before implementation.

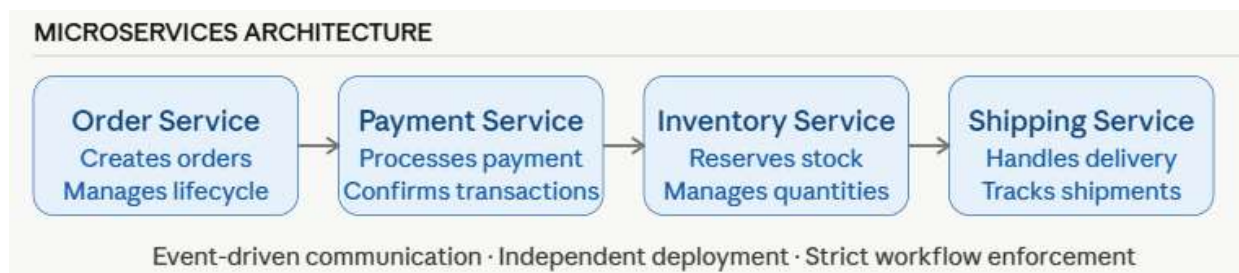
1.2 Microservices Concept

A microservices architecture is a software design approach where a system is divided into small independent services. Each service handles a specific task and communicates with others through events.

In this system:

- Order Service handles order creation
- Payment Service handles payment processing
- Inventory Service handles stock management
- Shipping Service handles delivery

Each service works independently but follows a strict event-driven workflow to maintain consistency.



1.3 Why Formal Verification is Needed

Traditional testing is not enough for complex distributed systems because it cannot check all possible states.

Formal verification is used because:

- » It mathematically proves system correctness
- » It detects hidden design errors early
- » It ensures no invalid states exist
- » It improves reliability and safety

In this project, formal methods ensure that:

- » Orders cannot be shipped without payment
- » Inventory cannot be reserved incorrectly
- » System state transitions remain valid

1.4 System Explanation

The system models an order lifecycle using states:



Each transition is controlled using formal rules. The system ensures that no invalid transitions occur, such as skipping payment or shipping cancelled orders.

2. OBJECTIVES

The main objectives of this project are:

- To design a formally verified microservices system
- To model system states using Z Notation
- To define functional behavior using VDM
- To detect structural issues using Alloy Analyzer
- To ensure correct event-driven workflow
- To identify and remove invalid system states
- To validate system correctness using CI pipeline

Formal Verification Goals:

- Ensure all order transitions are valid
- Prevent inconsistent system states
- Verify correctness mathematically
- Improve system reliability before implementation

3. METHODOLOGY

The project follows a structured SVV pipeline consisting of multiple stages.



3.1 Requirement Engineering

The first phase involves collecting and analyzing system requirements.

Functional Requirements:

- Create order
- Process payment
- Reserve inventory
- Ship order
- Cancel order

Non-Functional Requirements:

- System consistency
- Reliability
- Correct state transitions
- Verifiability

Requirement defects such as ambiguity and inconsistency were removed before formal modeling.

3.2 Z Notation Modeling

Z Notation is used to define the system mathematically.

It includes:

- System states
- System invariants
- State transitions

System States:

Created, Paid, Reserved, Shipped, Cancelled

Invariants:

- A shipped order must be paid
- Cancelled orders cannot be shipped
- Reserved orders must exist in system

Z modeling ensures that invalid system states are not possible.

3.3 VDM Specification

VDM defines system behavior using:

- Preconditions (conditions before operation)
- Postconditions (results after operation)

Operations:

- CreateOrder
- ConfirmPayment
- ReserveInventory
- ShipOrder
- CancelOrder

Each operation ensures correctness by enforcing rules such as:

- Payment must occur before reservation
- Reservation must occur before shipping

3.4 Alloy Analysis

Alloy Analyzer is used for structural verification.

It checks:

- Relationships between states
- Possible invalid scenarios
- Counterexamples in system design

Example Issue Found:

A scenario was detected where an order was reserved without payment.

This was corrected by adding proper constraints.

Alloy helps ensure that:

- No invalid order flow exists
- System structure remains consistent

3.5 CI Pipeline

GitHub Actions is used to automate validation.

It performs:

- Project structure verification
- File existence checking
- Consistency validation

This ensures that all SVV components are properly maintained and structured.

4. TOOLS USED

The following tools were used in this project:

- GitHub → Version control and repository management
- Z Notation (CZT) → Formal state modeling
- VDM (Overture Tool) → Functional specification
- Alloy Analyzer → Constraint verification
- GitHub Actions → CI pipeline automation

These tools collectively ensure formal correctness and validation.

FORMAL VERIFICATION TOOLS

Z Notation (CZT)
Mathematical state modeling
Defines invariants
Prevents invalid transitions
✓ All states verified

VDM (Overture)
Functional specification
Pre/post conditions
5 operations defined
✓ All operations correct

Alloy Analyzer
Constraint verification
Counterexample detection
Found: missing constraint
✓ Fixed & re-verified

5. RESULTS

The project produced the following results:

5.1 Z Notation Results

- System states were successfully defined
- Invariants ensured correct transitions
- No invalid state transitions allowed

5.2 VDM Results

- All operations were defined with correct pre/post conditions
- Functional behavior matched system requirements
- Order lifecycle followed correct sequence

5.3 Alloy Results

- Detected invalid scenario: order reserved without payment
- Identified missing constraints
- After correction, no invalid states were generated

5.4 CI Pipeline Results

- Repository structure validated successfully
- Required files verified
- Build process executed without errors

KEY RESULTS

✓
Valid state transitions

✓
No invalid states remain

✓
CI pipeline passed

✓
Correct event workflow

GITHUB REPOSITORY LINK

<https://github.com/uabdulrehman/Microservices-Order-Processing-System.git>

6. CONCLUSION

This project successfully demonstrates the use of Software Verification and Validation techniques for a Microservices Order Processing System.

The system was formally verified using Z Notation, VDM, and Alloy, ensuring correctness before implementation.

The verification process ensured:

- Correct order lifecycle
- No invalid state transitions
- Strong system consistency
- Reliable event-driven behavior

Overall, formal methods significantly improved system reliability and correctness by detecting issues that traditional testing cannot identify.